"Advanced Statistics for Physics Analysis"
Laboratory session 2 - report

Libero Pollini

May 26, 2026

Overview

We are given a dataset of integer-valued observations (in an attached file):

$$D = \{x_1, x_2, ..., x_N\} \tag{1}$$

representing the number of events observed in identical consecutive time intervals. We are told that the order of observations matters. Our goal is to analyze the data in the Bayesian framework using Markov chain Monte Carlo (MCMC) methods, with algorithms coded in native R. Throughout the assignment, we use the Gamma density parameterized by shape and rate,

$$p(\lambda|\alpha,\beta) = \text{Gamma}(\alpha,\beta) \equiv \frac{\beta^\alpha}{\Gamma(\alpha)} \cdot \lambda^{\alpha-1} \cdot e^{-\beta\lambda}, \quad \lambda > 0 \tag{2}$$

Unless otherwise stated, we use:

$$\alpha = 1, \quad \beta = 0.1 \tag{3}$$

A first visualization of the data is provided in figure 1, where we get an impression of an increase in event rate at some point in time between 50 and 75.

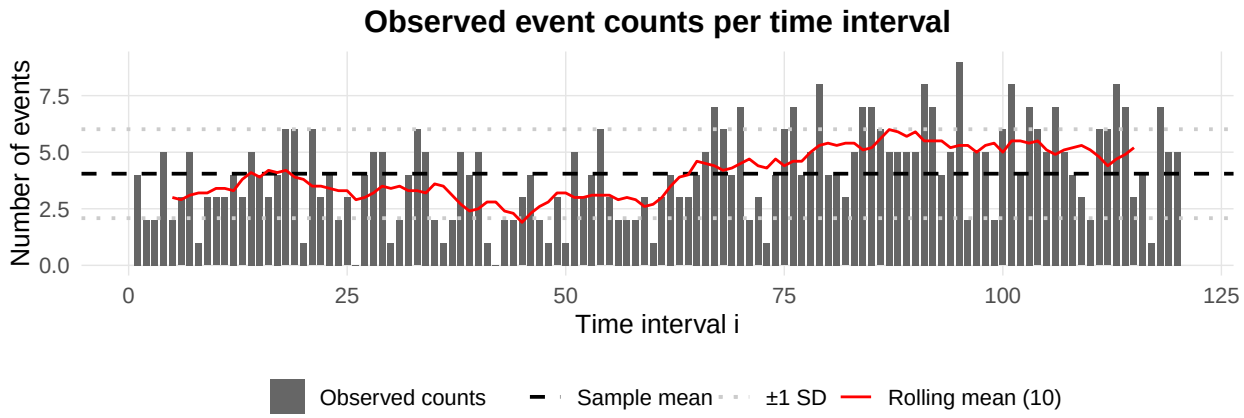


Figure 1: Observed event counts per time interval, with superposition of relevant statistics.

1 Bayesian change-point model

In all of the following, we *assume* that the data are generated from a Poisson model with one unknown change point. Before the change point, the event rate is λ_1; after the change point, the event rate is λ_2. More precisely,

$$x_i|\lambda_1, m \sim \text{Poisson}(\lambda_1), \quad i = 1, \cdots, m, \tag{4}$$

$$x_i|\lambda_2, m \sim \text{Poisson}(\lambda_2), \quad i = m+1, \cdots, N \tag{5}$$

where

$$m \in \mathcal{M} \equiv \{n_{\min}, n_{\min}+1, \cdots, N - n_{\min}\}, \tag{6}$$

with $n_{\min} = 5$.

We *choose* the prior distributions as

$$\lambda_1 \sim \text{Gamma}(\alpha, \beta) \tag{7}$$

$$\lambda_2 \sim \text{Gamma}(\alpha, \beta) \tag{8}$$

$$m \sim \text{Unif}(\mathcal{M}) \tag{9}$$

We recall that

$$x_i \sim \text{Poisson}(\lambda_1) \leftrightarrow p(x_i) = \frac{\lambda_1^{x_i} e^{-\lambda_1}}{x_i!} \tag{10}$$

Basing on these assumptions, the likelihood distribution is then (using the assumption of independent identically distributed samples, for each of the two intervals, and doing some trivial algebra):

$$p(D|\lambda_1, \lambda_2, m) = \prod_{i=1}^{m} \frac{\lambda_1^{x_i} e^{-\lambda_1}}{x_i!} \cdot \prod_{i=m+1}^{N} \frac{\lambda_2^{x_i} e^{-\lambda_2}}{x_i!} = \frac{\lambda_1^{S_m} \cdot \lambda_2^{S_N - S_m} \cdot e^{-(m\lambda_1 + (N-m)\lambda_2)}}{\prod_i x_i!} \tag{11}$$

where we defined the cumulative sums:

$$S_m \equiv \sum_{i=1}^{m} x_i, \quad S_N \equiv \sum_{i=1}^{N} x_i \tag{12}$$

The posterior distribution is then (using equations 11, 2 and Bayes' theorem, and doing some trivial algebra):

$$\begin{aligned} p(\lambda_1, \lambda_2, m|D) &\propto p(D|\lambda_1, \lambda_2, m) \cdot p(\lambda_1)p(\lambda_2)p(m) \propto \\ p(D|\lambda_1, \lambda_2, m) &\cdot p(\lambda_1)p(\lambda_2) \propto \\ \lambda_1^{S_m} \cdot \lambda_2^{S_N - S_m} \cdot e^{-(m\lambda_1 + (N-m)\lambda_2)} &\cdot \lambda_1^{\alpha-1} e^{-\beta\lambda_1} \cdot \lambda_2^{\alpha-1} e^{-\beta\lambda_2} = \\ \lambda_1^{S_m + \alpha - 1} \cdot \lambda_2^{S_N - S_m + \alpha - 1} &\cdot e^{-[(m+\beta)\lambda_1 + (N-m+\beta)\lambda_2]} \end{aligned} \tag{13}$$

where we defined the cumulative sums:

$$S_m \equiv \sum_{i=1}^{m} x_i, \quad S_N \equiv \sum_{i=1}^{N} x_i \tag{14}$$

Recalling the definition of Gamma distribution (2), we observe that equation 13 can be interpreted as:

$$\begin{aligned} p(\lambda_1|D, m) = \text{Gamma}(\alpha_1, \beta_1), \quad p(\lambda_2|D, m) = \text{Gamma}(\alpha_2, \beta_2) \\ \text{with } (\alpha_1, \beta_1, \alpha_2, \beta_2) \equiv (S_m + \alpha, \quad m + \beta, \quad S_N - S_m + \alpha, \quad N - m + \beta) \end{aligned} \tag{15}$$

That is, the unnormalized posterior, conditioned on m, can be factorized into two independent Gamma distributions.

Using Bayes' theorem (keeping fixed the conditioning on the lambdas) and the assumed independence of all model parameters on one another, we have (plugging in eq.s 11 and 9):

$$\begin{aligned} p(m|\lambda_1, \lambda_2, D) \propto p(D|m, \lambda_1, \lambda_2) \cdot p(m|\lambda_1, \lambda_2) = p(D|m, \lambda_1, \lambda_2) \cdot p(m) \propto \\ \lambda_1^{S_m} \cdot \lambda_2^{S_N - S_m} \cdot e^{-(m\lambda_1 + (N-m)\lambda_2)} \cdot \mathbf{1}_{\mathcal{M}} \propto \\ \left(\frac{\lambda_1}{\lambda_2}\right)^{S_m} \cdot e^{-(\lambda_1 - \lambda_2)m}, \quad m \in \mathbf{N} \cap [n_{\min}, N - n_{\min}] \end{aligned} \tag{16}$$

Which is of course a discrete distribution in m, as m is the discrete index of the change-point.

We observe that the joint posterior $p(\lambda_1, \lambda_2, m|D)$ has no standard closed form, so MCMC is the natural tool to sample from it. Furthermore, the full conditional distributions are all available analytically (Gamma for λ_1, λ_2, discrete for m), making a Gibbs sampler easy to implement.

2 Gibbs sampler

We now implement a Gibbs sampler for $(\lambda_1, \lambda_2, m)$. Conceptually, this simply corresponds to updating the parameters one at a time, sampling them from the distribution conditioned on all other parameters:

Algorithm 1 Gibbs sampler for change-point model

Set prior: $\alpha = 1$, $\beta = 0.1$, $n_{\min} = 5$
Initialize $\lambda_1^{(1)}, \lambda_2^{(1)}, m^{(1)}$
for $t = 2$ **to** T **do**
  $S_m \leftarrow \sum_{i=1}^{m^{(t-1)}} x_i$
  Sample $\lambda_1^{(t)} \sim \text{Gamma}\big(S_m + \alpha,\ m^{(t-1)} + \beta\big)$
  Sample $\lambda_2^{(t)} \sim \text{Gamma}\big((S_N - S_m) + \alpha,\ (N - m^{(t-1)}) + \beta\big)$
  for each $m \in \{n_{\min}, \dots, N - n_{\min}\}$ **do**
    Compute $p_m \propto \left(\frac{\lambda_1^{(t)}}{\lambda_2^{(t)}}\right)^{S_m} e^{-(\lambda_1^{(t)} - \lambda_2^{(t)})m}$
  end for
  Normalize p_m to sum to 1
  Sample $m^{(t)} \sim \text{Categorical}(\{p_m\})$
end for
return $\{(\lambda_1^{(t)}, \lambda_2^{(t)}, m^{(t)})\}_{t=1}^{T}$

Below (Figure 2) are the trace plots (with no burn-in implemented yet) we obtained for three independent arbitrarily initialized Gibbs sampler MC Markov chains, each with 1 000 000 steps, for each of the three parameters, from which we get the impression of a rather fast convergence.

In Figure 3 we also plot, for each chain, the autocorrelation function (which is built-in in R; we used a fixed maximum lag of 100 for the sake of visualization) for each chain and each parameter, after a burn-in period that we set at iteration 10 000. The fast decay to zero in all cases signals a good quality of the obtained chains.

In the following, we always consider the three chains without the burnt-in samples, merged together, for a total of $N_c = 3 \cdot (1\,000\,000 - 10\,000) = 2\,990\,000$ samples. For each parameter, we also compute the *effective sample size*. This is defined, in the case of a Gibbs sampler such as ours, as

$$\text{ESS} = \frac{N_c}{2\tau_{\text{int}}} \tag{17}$$

To obtain an estimate of the integrated autocorrelation time τ_{int} we decide to adopt Geyer's *initial monotone sequence* estimator [1]. That is, we compute the sample autocorrelation function $\rho_A(\tau)$ at lags $\tau = 1, 2, \dots, 1000$ (1000 is chosen for ease of computation) and form the pairs

$$P_k \equiv \rho_A(2k) + \rho_A(2k+1)\ , \qquad k = 1, 2, \dots \tag{18}$$

Let K be the first index such that $P_K < 0$; if all computed pairs are positive we take K as the maximum available number of pairs. The integrated autocorrelation time *for a single chain* is then

$$\tau_{\text{int}} = 1 + 2 \sum_{k=1}^{K-1} P_k\ , \tag{19}$$

In our cases, we get very small K indexes (below 10), hinting once again at the fact that the chains are converging fast. The total ESS is the sum of the ESS for each chain, each computed using the respective integrated correlation time. We obtain the following effective sample sizes:

Parameter	λ_1	λ_2	m
ESS	1 166 494	1 256 340	1 077 852

[1] See C. J. Geyer, *Practical Markov Chain Monte Carlo*, Statistical Science, 7(4):473–483, 1992.

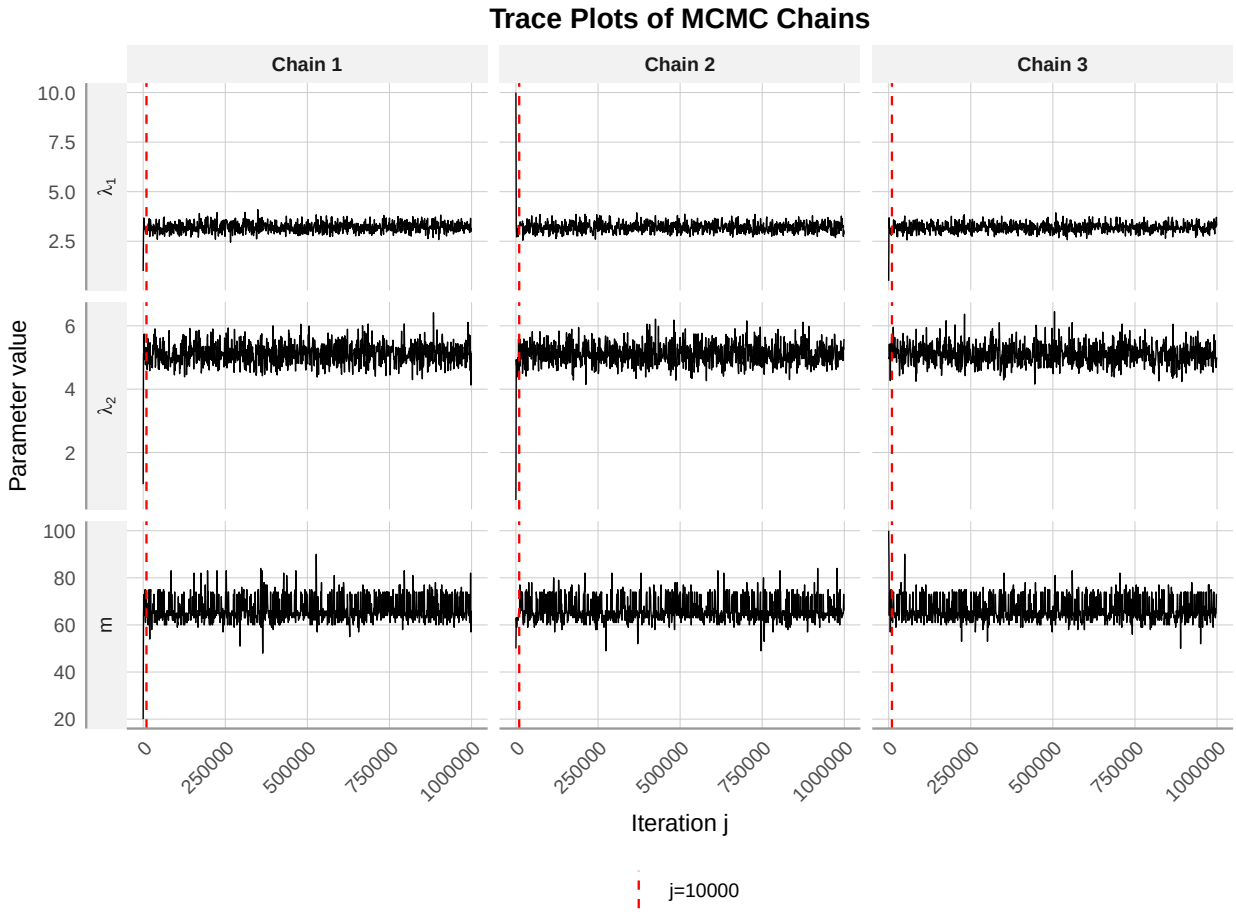


Figure 2: Trace plots (every 1000 iterations) for λ_1, λ_2, and m across three independent chains.

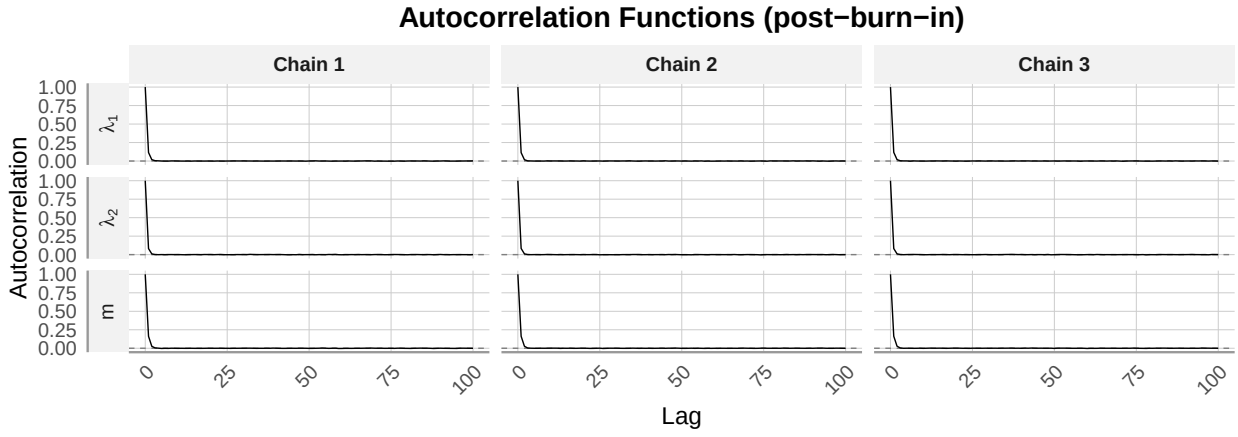


Figure 3: Autocorrelation functions for λ_1, λ_2, and m after burn-in (lag up to 100, burn-in 10 000). The dashed horizontal line marks zero autocorrelation.

Table 1 collects the posterior mean, median, standard deviation, MAP estimate (*i.e.*, the argmax of the PDF constructed by means of a point-by-point Gaussian Kernel Density Estimate with bandwidth set using Silverman's rule of thumb, for the continuous parameters λ_1 and λ_2; and the mode of the PMF, for the

discrete m parameter) and 95% credible interval for each parameter, based on the combined post-burn-in sample.

Table 1: Posterior summaries for λ_1, λ_2 and m.

Parameter	Mean	Median	SD	MAP	95% CI
λ_1	3.207	3.202	0.234	3.195	[2.763, 3.681]
λ_2	5.111	5.101	0.323	5.075	[4.504, 5.773]
m	66.2	65	5.1	65	[59, 77]

We can estimate the posterior probability that the rate increases after the change point rather simply, as the fraction of MC samples satisfying that condition:

$$p(\lambda_2 > \lambda_1 \mid D) \approx \frac{1}{N_c} \sum_{t=1}^{N_c} \mathbf{1}_{\lambda_2^{(t)} > \lambda_1^{(t)}} \approx 0.999996 \ ,$$

which is effectively 1, signaling that the for this given model, the rate almost certainly increases after the change-point.

3 Posterior predictive distribution

For a new observation x_{new} from the same process, the posterior predictive distribution is obtained by marginalizing over the joint posterior:

$$p(x_{\text{new}} \mid D) = \sum_{m \in \mathcal{M}} \int_0^{+\infty} d\lambda_1 \int_0^{+\infty} d\lambda_2 \; p(x_{\text{new}} \mid \lambda_1, \lambda_2, m)\, p(\lambda_1, \lambda_2, m \mid D) \tag{20}$$

For a future observation after time N, by definition of our model (in particular, where we assumed $m < N - n_{\mathbf{min}}$) we have that the relevant rate is λ_2.

Therefore, *for any time greater than N*, we have:

$$E[x_{\text{new}}|D] = E[\text{Poisson}(\lambda_2)] \approx E[\text{Poisson}(\hat{\lambda}_2)] = \hat{\lambda}_2 \approx 5.08 \tag{21}$$

(Where we use the well known expression for the mean of the Poisson distribution).

We now want to estimate the probability of a new sample being larger than $2\bar{x}$, with

$$\bar{x} \equiv \frac{1}{N} \sum_{i=1}^{N} x_i \tag{22}$$

(the empirical average of the sample). Once again, the relevant rate is λ_2, so

$$P(x_{\text{new}} > 2\bar{x}) \approx E_{\lambda_2}\big[P(X > 2\bar{x} \mid \lambda_2)\big] \equiv \frac{1}{N_c} \sum_{t=1}^{N_c} P\big(X > 2\bar{x} \;\big|\; \lambda_2^{(t)}\big), \qquad X \sim \text{Poisson}(\lambda_2^{(t)}) \tag{23}$$

Numerically, we obtain $P(x_{\text{new}} > 2\bar{x}) \approx 7.76\%$. Figure 4 shows the posterior predictive distribution of x_{new}. The vertical lines mark $\bar{x} \approx 4.05$ and $2\bar{x} \approx 8.10$.

4 Model comparison

We now want to compare the one-change-point modelM_1 with the simpler no-change model M_0,

$$M_0 : x_i|\lambda \sim \text{Poisson}(\lambda), \quad i = 1, \cdots, N \tag{24}$$

with the same prior $\lambda \sim \text{Gamma}(\alpha, \beta)$.

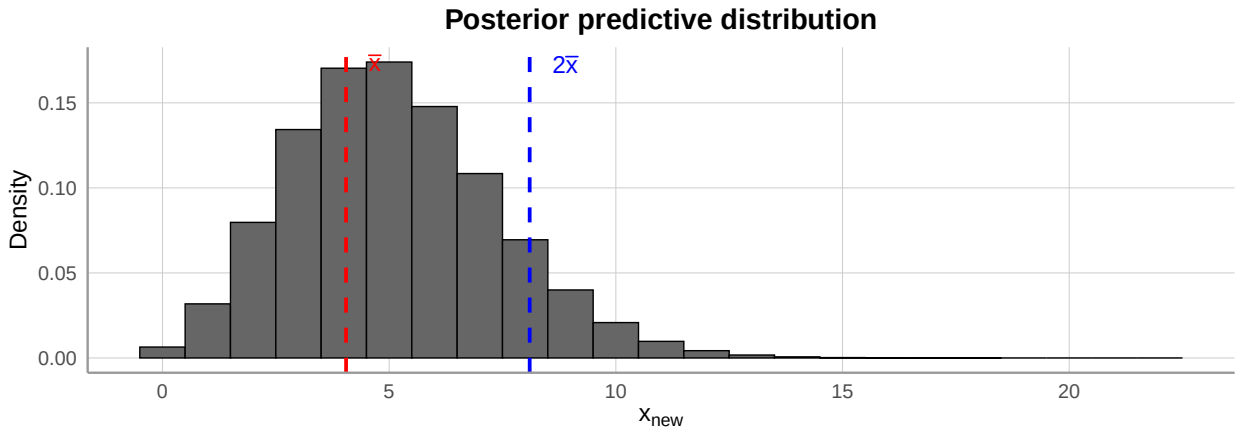


Figure 4: Posterior predictive distribution of x_{new} after time N (simulated).

The marginal likelihood for M_0 is

$$p(D|M_0) = \int_0^{+\infty} d\lambda \cdot p(D|\lambda) \cdot p(\lambda) = \int_0^{+\infty} d\lambda \cdot \prod_{i=1}^{N} \left(\frac{\lambda^{x_i} e^{-\lambda}}{x_i!} \right) \cdot \frac{\beta^\alpha}{\Gamma(\alpha)} \lambda^{\alpha-1} e^{-\beta\lambda} = \frac{1}{\prod_i (x_i!)} \frac{\beta^\alpha}{\Gamma(\alpha)} \frac{\Gamma(\alpha + S_N)}{(\beta + N)^{\alpha+S_N}} \tag{25}$$

(Where we used the normalization of the Gamma distribution). On the other hand, for the change-point model we have as before (equation 11):

$$\begin{aligned} p(D|M_1) = \sum_{m \in \mathcal{M}} \int_0^{+\infty} d\lambda_1 \int_0^{+\infty} d\lambda_2 \cdot \prod_{i=1}^{m} \frac{\lambda_1^{x_i} e^{-\lambda_1}}{x_i!} \cdot \prod_{i=m+1}^{N} \frac{\lambda_2^{x_i} e^{-\lambda_2}}{x_i!} \cdot p(\lambda_1, \lambda_2, m) = \\ \frac{1}{|\mathcal{M}|} \frac{1}{\prod_i (x_i!)} \left(\frac{\beta^\alpha}{\Gamma(\alpha)} \right)^2 \sum_{m \in \mathcal{M}} \int_0^{+\infty} d\lambda_1 \cdot \lambda_1^{S_m + \alpha - 1} e^{-(m+\beta)\lambda_1} \cdot \int_0^{+\infty} d\lambda_2 \lambda_2^{S_N - S_m + \alpha} e^{-(N-m+\beta)\lambda_2} = \\ \frac{1}{|\mathcal{M}|} \frac{1}{\prod_i (x_i!)} \left(\frac{\beta^\alpha}{\Gamma(\alpha)} \right)^2 \sum_{m \in \mathcal{M}} \left(\frac{\Gamma(S_m + \alpha)}{(\beta + m)^{S_m + \alpha}} \cdot \frac{\Gamma(S_N - S_m + \alpha)}{(\beta + N - m)^{S_N - S_m + \alpha}} \right) \end{aligned} \tag{26}$$

(where we used the same property and reordered the terms).

Therefore the Bayes factor simplifies to:

$$\begin{aligned} \text{BF}_{10} \equiv \frac{p(D|M_1)}{p(D|M_0)} = \\ \frac{\beta^\alpha}{\Gamma(\alpha)} \frac{(\beta + N)^{S_N + \alpha}}{\Gamma(S_N + \alpha)} \frac{1}{|\mathcal{M}|} \sum_{m \in \mathcal{M}} \frac{\Gamma(S_m + \alpha)}{(m + \beta)^{S_m + \alpha}} \frac{\Gamma(S_N - S_m + \alpha)}{(N - m + \beta)^{S_N - S_m + \alpha}} \end{aligned} \tag{27}$$

Numerically, we obtain $BF_{10} \approx 4.00 \cdot 10^3$.

5 Conclusion

The Bayes factor $\text{BF}_{10} \approx 4.00 \cdot 10^3$ provides decisive evidence for a change point (M_1) over the constant-rate model (M_0). The most plausible location is $m = 65$ (MAP, Table 1), with a 95% credible interval $[59, 77]$. The two rates are clearly separated: λ_1 (posterior mean 3.21) and λ_2 (5.11); the probability $\Pr(\lambda_2 > \lambda_1 \mid D) \approx 1$. Trace plots (Figure 2) and autocorrelation functions (Figure 3) show rapid mixing and consistent behaviour across all three chains. Autocorrelations decay to zero within lag < 10; the effective sample sizes ($\sim 10^6$ per parameter) are large relative to the $2.997 \cdot 10^6$ post-burn-in draws, indicating reliable MCMC estimates. We observe that Hamiltonian Monte Carlo cannot be applied directly because m is discrete, making the joint state space non-differentiable; one must marginalize m out or use a discrete sampler (e.g., Gibbs).

Code

The complete R code (for all sections) follows, commented where necessary. Output, where needed, is also included as comments.

```r
library(ggplot2)
library(zoo)  # for rollmean

# import dataset
full_data <- read.csv("C:/Users/Libero/uni/1B/adv_stats/assignment_2/assignment2_changepoint_counts.csv", header=TRUE)
data <- full_data$x
# explore it
typeof(data)
#[1] "integer"
s_N <- sum(data)
cat(s_N)
#486
N <- length(data)
cat(N)
#120
x_mean <- mean(data)
cat(x_mean)
#4.05
x_std <- sd(data)
cat(x_std)
#1.965458

# barplot of data + statistics
df_bar <- data.frame(i = seq_along(data), x = data)
df_bar$rollmean <- rollmean(data, k = 10, fill = NA, align = "right")

ggplot(df_bar, aes(x = i, y = x)) +
  geom_col(aes(fill = "Observed counts"), width = 0.8) +
  geom_hline(aes(yintercept = x_mean, color = "Sample mean", linetype = "Sample mean"),
             linewidth = 0.8) +
               geom_hline(aes(yintercept = x_mean + x_std, color = "\u00b11 SD", linetype = "\u00b11 SD"),
             linewidth = 0.8) +
  geom_hline(aes(yintercept = x_mean - x_std, color = "\u00b11 SD", linetype = "\u00b11 SD"),
             linewidth = 0.8) +
  geom_line(aes(y = rollmean, color = "Rolling mean (10)", linetype = "Rolling mean (10)"),
            linewidth = 0.6, na.rm = TRUE) +
  scale_fill_manual(name = NULL, values = c("Observed counts" = "grey40")) +
  scale_color_manual(name = NULL,
                     breaks = c("Observed counts", "Sample mean", "\u00b11 SD", "Rolling mean (10)"),
                     values = c("Observed counts" = "grey40",
                                "Sample mean" = "black",
                                "\u00b11 SD" = "grey80",
                                "Rolling mean (10)" = "red")) +
scale_linetype_manual(name = NULL,
                      breaks = c("Observed counts", "Sample mean", "\u00b11 SD", "Rolling mean (10)"),
                      values = c("Observed counts" = "solid",
                                 "Sample mean" = "dashed",
```

```
                                  "\u00b11 SD" = "dotted",
                                  "Rolling mean (10)" = "solid")) +
  labs(title = "Observed event counts per time interval",
       x = "Time interval i", y = "Number of events") +
  theme_minimal(base_size = 12) +
  theme(panel.grid.major = element_line(color = "grey90", linewidth = 0.3),
        panel.grid.minor = element_blank(),
        plot.title = element_text(face = "bold", hjust = 0.5),
        legend.position = "bottom")


############################################################
#                    GIBBS SAMPLER
############################################################

# pick prior parameters
alpha <- 1
beta <- 0.1

gibbs_sampler <- function(n_steps, counts, start, alpha, beta, m_min, m_max) {
  lambda_1 <- numeric(n_steps)
  lambda_2 <- numeric(n_steps)
  m   <- numeric(n_steps)
  lambda_1[1] <- start[1]
  lambda_2[1] <- start[2]
  m[1]   <- start[3]

  N <- length(counts)
  s_N <- sum(counts)                      # constant

  # Allowed m values
  m_vals <- m_min:m_max
  log_probs <- numeric(length(m_vals))    # to avoid recomputing full pmf

  for (t in 2:n_steps) {
    m_curr <- m[t-1]
    s_m    <- sum(counts[1:m_curr])       # sum of first m_curr observations

    # Gibbs step 1: lambda_1
    alpha1 <- s_m + alpha
    beta1  <- m_curr + beta
    lambda_1[t] <- rgamma(1, shape = alpha1, rate = beta1)

    # Gibbs step 2: lambda_2
    alpha2 <- s_N - s_m + alpha
    beta2  <- N - m_curr + beta
    lambda_2[t] <- rgamma(1, shape = alpha2, rate = beta2)

    # Gibbs step 3: m
    # Precompute S_m for all candidate m
    S_cand <- cumsum(counts)[m_vals]      # cumulative sum up to each candidate
    log_probs <- S_cand * (log(lambda_1[t]) -log( lambda_2[t])) +
                 m_vals * (lambda_2[t] - lambda_1[t])
    probs <- exp(log_probs - max(log_probs))  # stabilise: by subtracting a
        constant get one exponent = 0, to avoid division by 0; we normalize after
        anyways
    probs <- probs / sum(probs) # normalization
    m[t] <- sample(m_vals, size = 1, prob = probs)
  }
```

```
105
106   cbind(lambda1 = lambda_1, lambda2 = lambda_2, m = m)
107 }
108
109 N_tot=1000000
110
111 system.time({
112 chains <- list(
113   gibbs_sampler(N_tot, data, c(1, 1, 20), alpha, beta, 5, N-5), # arbitrary
        starting point
114   gibbs_sampler(N_tot, data, c(10, 0.5, 50), alpha, beta, 5, N-5),
115   gibbs_sampler(N_tot, data, c(0.5, 5, 100), alpha, beta, 5, N-5)
116 )})
117 # user  system elapsed
118 #    user  system elapsed
119 # 130.22    0.89  141.78
120
121 # Combine the three chains into a single data frame for plotting - keep only every
      1000 iterations for faster plotting
122 # 1. For each chain i, turn its matrix into a data frame with iteration number,
     parameters, and a chain label
123 chain_df <- do.call(rbind, lapply(seq_along(chains), function(i) {
124   full_n <- nrow(chains[[i]])
125   keep_idx <- seq(1, full_n, by = 1000)          # indices to keep
126   mat <- chains[[i]][keep_idx, , drop = FALSE]   # extract those rows
127   data.frame(iteration = keep_idx,
128              lambda1   = mat[, "lambda1"],
129              lambda2   = mat[, "lambda2"],
130              m         = mat[, "m"],
131              chain     = factor(i))
132 }))
133
134 burn_in  <- 10000 # for now, only used to show vertical line
135
136 # Reshape to long format for faceting
137 chain_long <- with(chain_df, data.frame(
138   iteration = rep(iteration, 3),
139   value = c(lambda1, lambda2, m),
140   parameter = rep(c("lambda[1]", "lambda[2]", "m"), each = nrow(chain_df)),
141   chain = factor(rep(chain, 3), labels = paste("Chain", 1:3))
142 ))
143
144 ggplot(chain_long, aes(x = iteration, y = value)) +
145   geom_line(color = "black", linewidth = 0.3) +
146   geom_vline(aes(xintercept = burn_in, color = "j=10000", linetype = "j=10000"),
147              linewidth = 0.5, show.legend = TRUE) +
148   facet_grid(rows = vars(parameter), cols = vars(chain),
149              scales = "free_y", switch = "y",
150              labeller = labeller(parameter = label_parsed, chain = label_value)) +
151   scale_color_manual(name = NULL, values = c("j=10000" = "red")) +
152   scale_linetype_manual(name = NULL, values = c("j=10000" = "dashed")) +
153   labs(x = "Iteration j", y = "Parameter value") +
154   ggtitle("Trace Plots of MCMC Chains") +
155   theme_minimal(base_size = 11) +
156   theme(
157     panel.grid.major = element_line(color = "grey80", linewidth = 0.2),
158     panel.grid.minor = element_blank(),
159     strip.background = element_rect(fill = "grey95", color = NA),
160     strip.text = element_text(face = "bold"),
```

```r
    axis.line = element_line(color = "grey60"),
    plot.title = element_text(face = "bold", hjust = 0.5),
    axis.text.x = element_text(angle = 45, hjust = 1),
    legend.position = "bottom"
  )

# ================================================
# ================================================
# PARAMETER ESTIMATION
# ================================================
# ================================================

lag_max  <- 1000
# Concatenated chain (used later for summaries and acf plots)
chain <- do.call(rbind, lapply(chains, function(ch) ch[-(1:burn_in), ]))
N_c   <- nrow(chain)            # total  p o s t burnin  draws (optional)
cat(N_c)
# 2970000

# ---------------------------------------------------------
# 1.  Parameter lambda1
# ---------------------------------------------------------
ESS_lambda1 <- 0
for (i in seq_along(chains)) { #
    ch_burned <- chains[[i]][-(1:burn_in), "lambda1"]
    acf_obj   <- acf(ch_burned, plot = FALSE, lag.max = lag_max)
    rho       <- acf_obj$acf[-1]
    pairs     <- rho[seq(1, length(rho)-1, by = 2)] + rho[seq(2, length(rho), by =
        2)]
    pos       <- which(pairs < 0)[1]
    if (is.na(pos)) pos <- length(pairs) + 1
    tau_int   <- 1 + 2 * sum(pairs[1:(pos-1)])
    ESS_lambda1 <- ESS_lambda1 + length(ch_burned) / (2 * tau_int)
}
cat("lambda1: ESS =", round(ESS_lambda1, 1), "\n")
# 1166494

# ---------------------------------------------------------
# 2.  Parameter lambda2
# ---------------------------------------------------------
ESS_lambda2 <- 0
for (i in seq_along(chains)) {
    ch_burned <- chains[[i]][-(1:burn_in), "lambda2"]
    acf_obj   <- acf(ch_burned, plot = FALSE, lag.max = lag_max)
    rho       <- acf_obj$acf[-1]
    pairs     <- rho[seq(1, length(rho)-1, by = 2)] + rho[seq(2, length(rho), by =
        2)]
    pos       <- which(pairs < 0)[1]
    if (is.na(pos)) pos <- length(pairs) + 1
    tau_int   <- 1 + 2 * sum(pairs[1:(pos-1)])
    ESS_lambda2 <- ESS_lambda2 + length(ch_burned) / (2 * tau_int)
}
cat("lambda2: ESS =", round(ESS_lambda2, 1), "\n")
# lambda2: ESS = 1256340
# ---------------------------------------------------------
# 3.  Parameter m
# ---------------------------------------------------------
ESS_m <- 0
for (i in seq_along(chains)) {
```

```r
    ch_burned <- chains[[i]][-(1:burn_in), "m"]
    acf_obj   <- acf(ch_burned, plot = FALSE, lag.max = lag_max)
    rho       <- acf_obj$acf[-1]
    pairs     <- rho[seq(1, length(rho)-1, by = 2)] + rho[seq(2, length(rho), by =
        2)]
    pos       <- which(pairs < 0)[1]
    if (is.na(pos)) pos <- length(pairs) + 1
    tau_int   <- 1 + 2 * sum(pairs[1:(pos-1)])
    ESS_m     <- ESS_m + length(ch_burned) / (2 * tau_int)
}
cat("m: ESS =", round(ESS_m, 1), "\n")
# m: ESS = 1077852


# ACF plot with faceting (same style as trace plots)
lag_max_acf <- 100
acf_list <- list()
for (i in seq_along(chains)) {
  ch <- chains[[i]][-(1:burn_in), ]
  for (par in c("lambda1", "lambda2", "m")) {
    acf_obj <- acf(ch[, par], lag.max = lag_max_acf, plot = FALSE)
    df_acf <- data.frame(
      lag = acf_obj$lag[,1,1],
      acf = acf_obj$acf[,1,1],
      chain = paste("Chain", i),
      parameter = switch(par,
                         lambda1 = "lambda[1]",
                         lambda2 = "lambda[2]",
                         m = "m")
    )
    acf_list[[length(acf_list) + 1]] <- df_acf
  }
}
acf_long <- do.call(rbind, acf_list)
acf_long$parameter <- factor(acf_long$parameter,
                             levels = c("lambda[1]", "lambda[2]", "m"))

ggplot(acf_long, aes(x = lag, y = acf)) +
  geom_hline(yintercept = 0, linetype = "dashed", colour = "grey50", linewidth =
      0.3) +
  geom_line(colour = "black", linewidth = 0.3) +
  facet_grid(rows = vars(parameter), cols = vars(chain),
             scales = "free_y", switch = "y",
             labeller = labeller(parameter = label_parsed)) +
  labs(x = "Lag", y = "Autocorrelation") +
  ggtitle("Autocorrelation Functions (post-burn-in)") +
  theme_minimal(base_size = 11) +
  theme(
    panel.grid.major = element_line(colour = "grey80", linewidth = 0.2),
    panel.grid.minor = element_blank(),
    strip.background = element_rect(fill = "grey95", colour = NA),
    strip.text = element_text(face = "bold"),
    axis.line = element_line(colour = "grey60"),
    plot.title = element_text(face = "bold", hjust = 0.5),
    axis.text.x = element_text(angle = 45, hjust = 1),
    legend.position = "bottom"
  )


```

```r
# ======================================================
# Posterior summaries for lambda1, lambda2, m
# ======================================================

# Helper function to compute MAP for continuous parameters

map_kde <- function(x) {
    dens <- density(x[seq(1, length(x))]) # compute a kernel density estimate (default: Gaussian basis,
                    # one curve per point, bandwith given by 1.06 * estimate(sigma) * sample_size)...
    dens$x[which.max(dens$y)]
}
# --- lambda1 ---
mean_l1 <- mean(chain[, "lambda1"])
med_l1  <- median(chain[, "lambda1"])
sd_l1   <- sd(chain[, "lambda1"])
map_l1  <- map_kde(chain[, "lambda1"])
ci_l1   <- quantile(chain[, "lambda1"], probs = c(0.025, 0.975))
# --- lambda2 ---
mean_l2 <- mean(chain[, "lambda2"])
med_l2  <- median(chain[, "lambda2"])
sd_l2   <- sd(chain[, "lambda2"])
map_l2  <- map_kde(chain[, "lambda2"])
ci_l2   <- quantile(chain[, "lambda2"], probs = c(0.025, 0.975))
# --- m (discrete: MAP is the most frequent value) ---
mean_m <- mean(chain[, "m"])
med_m  <- median(chain[, "m"])
sd_m   <- sd(chain[, "m"])
# MAP for discrete m: value with highest frequency
m_counts <- table(chain[, "m"])
map_m   <- as.numeric(names(m_counts)[which.max(m_counts)])
ci_m    <- quantile(chain[, "m"], probs = c(0.025, 0.975))
# Print results
cat(sprintf("lambda1: mean=%.3f, median=%.3f, sd=%.3f, MAP=%.3f, 95%% CI=[%.3f, %.3f]\n",
            mean_l1, med_l1, sd_l1, map_l1, ci_l1[1], ci_l1[2]))
# lambda1: mean=3.207, median=3.202, sd=0.234, MAP=3.190, 95% CI=[2.763, 3.681]
cat(sprintf("lambda2: mean=%.3f, median=%.3f, sd=%.3f, MAP=%.3f, 95%% CI=[%.3f, %.3f]\n",
            mean_l2, med_l2, sd_l2, map_l2, ci_l2[1], ci_l2[2]))
# lambda2: mean=5.111, median=5.101, sd=0.323, MAP=3.190, 95% CI=[4.504, 5.773]
cat(sprintf("m:       mean=%.1f, median=%.0f, sd=%.1f, MAP=%.0f, 95%% CI=[%.0f, %.0f]\n",
            mean_m, med_m, sd_m, map_m, ci_m[1], ci_m[2]))
# m:      mean=66.2, median=65, sd=5.1, MAP=65, 95% CI=[59, 77]

# ======================================================
# Probability that lambda2 > lambda1
# ======================================================
prob_l2_gt_l1 <- mean(chain[, "lambda2"] > chain[, "lambda1"])
cat(cat("P(lambda2 > lambda1) = ", prob_l2_gt_l1))
# P(lambda2 > lambda1) =  0.999996

# ======================================================
# Probability that x_new > 2 x_bar
# ======================================================
x_bar <- x_mean   # already computed
prob_new_gt_2xbar <- mean(ppois(2 * x_bar, lambda = chain[, "lambda2"], lower.tail
```

```
     = FALSE)) #
cat(sprintf("P(x_new > 2 x_bar | D) = %.4f\n", prob_new_gt_2xbar))
#P(x_new > 2 x_bar | D) = 0.0776
# plot Posterior predictive distribution
x_new_sim <- rpois(length(chain[, "lambda2"]), lambda = chain[, "lambda2"])
pred_df <- data.frame(x_new = x_new_sim)
ggplot(pred_df, aes(x = x_new)) +
  geom_histogram(aes(y = after_stat(density)), binwidth = 1, boundary = 0.5,
                 fill = "grey40", color = "black", linewidth = 0.3) +
  geom_vline(xintercept = x_bar, color = "red", linewidth = 0.8, linetype = "
      dashed") +
  geom_vline(xintercept = 2 * x_bar, color = "blue", linewidth = 0.8, linetype = "
      dashed") +
  annotate("text", x = x_bar + 0.5, y = Inf, label = "bar(x)",
           color = "red", hjust = 0, vjust = 1.5, parse = TRUE) +
  annotate("text", x = 2 * x_bar + 0.5, y = Inf, label = "2*bar(x)",
           color = "blue", hjust = 0, vjust = 1.5, parse = TRUE) +
  labs(title = "Posterior predictive distribution",
       x = expression(x[new]), y = "Density") +
  theme_minimal(base_size = 11) +
  theme(
    panel.grid.major = element_line(color = "grey80", linewidth = 0.2),
    panel.grid.minor = element_blank(),
    plot.title = element_text(face = "bold", hjust = 0.5),
    axis.line = element_line(color = "grey60")
  )

# ========================================================
# BAYES FACTOR
# ========================================================
m_min=5
m_max=N-5
# Compute Bayes factor BF10 = p(D | M1) / p(D | M0) using conjugate Gamma-Poisson
    marginals
# Already available: data, N, s_N, alpha, beta, m_min, m_max
                            # total sum of counts
m_vals <- m_min:m_max                    # allowed change points (5 to N-5)
S_m <- cumsum(data)[m_vals]              # cumulative sums up to each candidate m
# Log of the factor coming from M0's marginal (without the 1/prod(x!) factor,
    which cancels)
log_term0 <- alpha * log(beta) - lgamma(alpha) + (s_N + alpha) * log(N + beta) -
    lgamma(s_N + alpha)
# Log of the summand inside the sum over m (each term = product of two Gamma-
    Poisson marginals)
log_terms <- lgamma(S_m + alpha) - (S_m + alpha) * log(m_vals + beta) +
             lgamma(s_N - S_m + alpha) - (s_N - S_m + alpha) * log(N - m_vals +
                beta)
# Stable log-sum-exp across m
max_log <- max(log_terms)
log_sum <- log(sum(exp(log_terms - max_log))) + max_log - log(length(m_vals))
log_BF10 <- log_term0 + log_sum
cat(sprintf("log(BF10) = %.2f\n", log_BF10))
# log(BF10) = 8.29
cat(sprintf("BF10 = %.2e\n", exp(log_BF10)))
# BF10 = 4.00e+03
```

Declaration: AI assistance was used solely for improving the appearance of plots (no code generation or analysis).